

Lecture Name	Duration	Rating (/10)	Strategic Relevance for Infosys Round 2
Arrays - Lecture_1 (Introduction to Arrays)	00:20:51	3/10	Elementary array theory; skip if you understand 0-indexed memory.
Arrays-Lecture_2 (Array Traversal & Basics)	00:52:00	4/10	Basic for loops and element visits; low return on time investment.
Arrays - Lecture_3 (Array Manipulation Techniques)	00:31:52	7/10	Useful in-place pointer manipulation patterns for Q1.
Arrays - Lecture_4 (Prefix Sum)	00:37:15	10/10	Essential: Core $\mathcal{O}(N)$ template for continuous range sum queries.
Arrays - Lecture_5 (Prefix Sum Questions)	00:21:48	9/10	High Priority: Direct problem-solving templates for Q1 subarray checks.
Arrays - Lecture_6 (Prefix Sum Questions)	00:37:36	8/10	Covers negative values and boundary handling in prefix logic.
Arrays - Lecture_7 (Kadane's Algorithm)	00:26:39	10/10	Essential: Standard $\mathcal{O}(N)$ template for maximum subarray problems.

Arrays - Lecture_8 (Time Complexity of Array Operations)	00:31:41	9/10	Critical to understand time limits and prevent 10^5 nested loop TLEs.
Arrays - Lecture_9 (Most asked Interview Questions)	00:12:43	7/10	Quick revision of common interview-style array sweeps.
Arrays - Lecture_10 (Most asked Interview Questions)	00:32:45	7/10	Solid array interview pattern walkthroughs.
Linked List Lecture1 (Intro, Types, Creation)	01:01:11	2/10	Skip: Infosys OA does not test custom pointer linked lists.
Linked List Lecture2 (Insertion, Deletion, Traversal)	01:03:37	2/10	Skip: Heavy boilerplate code; not needed for array/string OAs.
Linked List Lecture3 (Interview Asked Practice Questions)	01:04:34	2/10	Skip: Low probability of appearing in an automated online round.
LinkedList Lecture4: Searching (Linear, Binary)	00:59:40	5/10	Binary search logic is good, but linked list context adds useless overhead.
LinkedList Lecture5: Sorting (Recursive Binary, Bubble)	01:03:45	2/10	Bubble sort triggers instant TLE in proctored tests; skip.

LinkedList Lecture6: Sorting (Selection, Insertion)	01:02:34	2/10	$\mathcal{O}(N^2)$ sorting methods; unusable for large constraints.
LinkedList Lecture7: Sorting (Merge Sort)	01:02:42	6/10	$\mathcal{O}(N \log N)$ concepts are useful, but Python .sort() handles this natively.
LinkedList Lecture8: Sorting (Quick Sort)	01:00:06	5/10	Theoretical sorting; rarely tested from scratch in online coding rounds.
LinkedList Lecture9: Sorting (Heap Sort)	00:38:25	5/10	Heap theory is useful, but building it via linked lists is counterproductive.
LinkedList Lecture10: Sorting (Heap Sort)	00:44:11	4/10	Redundant; standard Python heapq module is all you need.
String Lec-1 (Introduction to Strings)	00:40:00	3/10	String basics and syntax; skip if comfortable with slicing.
String Lec-2 (String Manipulation)	00:33:54	6/10	Useful for string replacement and basic parsing.
String Lec-3 (String Searching and Comparison)	00:29:09	6/10	Substring match logic; useful context for string lookups.

String Lec-4 (Frequency & Character Count Problem)	00:26:30	10/10	Essential: Core hash-based character counting for Q1.
String Lec-5 (Easy-Medium Level String Problems)	00:33:17	8/10	Solid practice for string-based Easy/Medium assessment questions.
String Lec-6 (Medium Level – Real Company Questions)	00:34:01	9/10	High Priority: Directly reflects actual company hiring assessments.
String Lec-7 (Medium Level – Real Company Questions)	00:19:02	8/10	High-yield practice for real-world medium string patterns.
Hashing: Introduction to Hashing Lec18	00:32:45	9/10	Essential: Foundational $O(1)$ lookup mechanics.
Hashing: Hashset Based Problems Lec19	00:35:40	8/10	Great for unique element tracking and instant presence verification.
Hashing: Hashmap for Strings & Pairs Lec20	00:48:58	10/10	Essential: The Two-Sum/pair lookup template needed for Q1.

Hashing: Important Interview Questions - PYQ1 Lec21	00:27:45	8/10	Direct previous year interview question practice.
Hashing: Important Interview Questions - PYQ2 Lec22	00:52:06	8/10	Reinforces pair-matching and frequency lookups.
Recursion1- (Introduction to Recursion)	00:49:21	4/10	Call stack theory; low practical utility for speed coding.
Recursion2- (Classic Recursive Problems)	00:29:37	5/10	Standard factorial/Fibonacci; skip if you know base cases.
Recursion3- (Introduction to Backtracking)	00:39:29	9/10	High Priority: Key template for brute-force Q3 scavenger points (\$N \le 20\$).
Recursion4- (Backtracking Part 2)	01:26:48	8/10	Generates all permutations/combinations for partial test case farming.
Recursion5- (Problem Solving-1)	00:58:16	6/10	Recursive tree exploration; moderate utility.
Recursion6- (Problem Solving-2)	00:36:21	6/10	Backtracking practice problems.

Recursion7- (Problem Solving-3)	00:38:53	5/10	Diminishing returns given the limited preparation window.
Stack&Queue-3 (Introduction to Queue)	01:03:38	4/10	FIFO mechanics; standard Python collections.deque covers this.
Stack&Queue-2 (Advanced Stack)	00:54:35	7/10	Monotonic stack patterns (Next Greater Element) sometimes appear in Q2.
Stack&Queue-1 (Introduction to Stack)	00:40:30	5/10	Basic LIFO principles; simple Python list .append() / .pop().
Stack&Queue-4 (Advanced Queue)	00:38:56	5/10	Sliding window queue mechanics.
Stack&Queue-5 (Problem Solving PART1)	00:23:19	5/10	General stack/queue problems; lower likelihood in OA.
Stack&Queue-6 (Problem Solving PART2)	00:30:23	5/10	Standard practice; low priority for target score.
Stack&Queue-7 (Problem Solving PART3)	00:27:56	5/10	Practice set; skip to focus on high-yield array/greedy topics.
Trees-1 (Introduction to Trees & Binary Tree)	00:46:17	3/10	Skip: Explicit tree construction rarely tested in online coding rounds.

Trees-2 (Tree Representation In Python)	00:39:43	3/10	Skip: Node class boilerplate is time-consuming in raw text editors.
Trees-3 (Tree Traversal - Inorder, Preorder, Postorder)	00:39:28	4/10	Traversal theory; lower priority for 2-solve target.
Trees-5 (BST Operations)	01:09:43	3/10	Skip: BST insertion/deletion boilerplate.
Trees-4 (Introduction to Binary Search Tree)	00:47:50	3/10	Skip: Low representation in standard hiring assessments.
Trees-8 (Problem Solving-2)	00:41:56	3/10	Tree problem sets; skip.
Trees-7 (Problem Solving-3)	00:37:34	3/10	Tree problem sets; skip.
Trees-6 (Problem Solving-1)	00:27:51	3/10	Tree problem sets; skip.

Graphs-1 (Introduction to Graphs)	00:58:17	4/10	Adjacency list/matrix representation theory.
Graphs2- (BFS)	00:42:19	6/10	Useful primarily if a 2D grid matrix shortest-path appears in Q2.
Graph3 - (DFS)	01:11:10	7/10	Focus exclusively on 2D grid/matrix flood-fill and island counting.
Graphs4- (Cycle Detection in Graphs)	01:15:28	4/10	Too complex and low yield for a 2-solve threshold.
Greedy1: (Introduction to Greedy Approach)	00:57:32	7/10	Solid conceptual foundation for greedy choice properties.
Greedy2: (Sorting-based Greedy Patterns)	00:48:53	10/10	Essential: Must-watch for Q2 interval and activity selection.
Greedy4: (Greedy in Graphs (Intro Level))	00:44:40	4/10	MST/Kruskal algorithms; skip for this round.
Greedy3: (Greedy for Strings & Numbers)	00:52:54	9/10	High Priority: Maps directly to adjacent lamp/character replacement sweeps.

DP1: (Introduction to Dynamic Programming)	00:52:07	6/10	Foundational memoization vs tabulation logic.
DP2: (DP on 1D Array – Starter Problems)	00:32:34	8/10	Good for 1D state transition problems using variables (prev1, prev2).
DP3: (DP on Strings (Intro))	00:52:42	5/10	LCS/Edit distance; high debugging overhead under proctored pressure.
DP4: (DP Optimization Basics)	00:57:24	4/10	Advanced space optimization; skip to focus on Q1/Q2 locks.